

LAPORAN PRAKTIKUM
PEMOGRAMAN BERORIENTASI OBJEK
“CLASS, OBJECT, CONSTRUCTOR dan METHOD”

disusun Oleh:
Sofian Arba'i (2511533029)

Dosen Pengampu:
Nurfiah, S.ST, M.Kom.

Asisten Praktikum:
M Dawi Syauqi



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Allah SWT atas rahmat dan karunia-Nya sehingga laporan praktikum Pemograman Berorientasi Objek dengan topik *class*, *objek*, *constructor*, dan *method* ini dapat diselesaikan dengan baik. Shalawat serta salam semoga senantiasa tercurahkan kepada Nabi Muhammad SAW.

Laporan ini disusun sebagai bentuk pertanggungjawaban atas pelaksanaan praktikum Pemograman Berorientasi Objek yang membahas konsep dasar class dan objek, pembuatan konstruktor, serta implementasi method menggunakan bahasa pemrograman Java.

Penyusunan laporan ini tidak terlepas dari bantuan, bimbingan, dan dukungan berbagai pihak. Oleh karena itu, saya ingin mengucapkan terima kasih yang sebesar-besarnya kepada:

1. Dosen pengampu mata kuliah Praktikum Pemograman Berorientasi Objek yang telah memberikan ilmu, arahan, serta bimbingannya.
2. Asisten praktikum yang telah membimbing dan memberikan penjelasan selama kegiatan praktikum berlangsung.

Saya meminta maaf jika masih terdapat kesalahan pada penulisan laporan ini. Oleh karena itu, saya menerima kritik dan saran yang membangun demi perbaikan di masa mendatang.

Padang, 10 September 2026

Sofian Arba'i

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan.....	1
1.3 Manfaat.....	2
BAB II PEMBAHASAN.....	3
2.1 Pengertian Class, Object, Constructor dan Method.....	3
2.1.1 Class	3
2.1.2 Object	3
2.1.3 Constructor dan Atribut	4
2.1.4 Method.....	5
2.2 Implementasi Class, Object, Constructor dan Method.....	5
2.2.1 Class Rekening	5
2.2.2 Class Main	7
BAB III PENUTUP.....	11
3.1 Kesimpulan.....	11
3.2 Saran	11
DAFTAR PUSTAKA	12

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi saat ini menuntut setiap mahasiswa untuk memahami konsep pemrograman secara mendalam, tidak hanya dari sisi sintaks bahasa pemrograman, tetapi juga dari sisi paradigma yang digunakan dalam membangun sebuah perangkat lunak. Salah satu paradigma yang paling banyak digunakan dalam pengembangan perangkat lunak modern adalah Pemrograman Berorientasi Objek (Object Oriented Programming/OOP), yang memandang suatu sistem sebagai kumpulan objek yang saling berinteraksi, bukan sekadar sekumpulan fungsi dan prosedur seperti pada paradigma pemrograman terstruktur.

Konsep dasar dalam OOP yang wajib dikuasai oleh setiap mahasiswa antara lain class, object, constructor, dan method. Class berperan sebagai cetak biru atau kerangka dasar yang mendefinisikan atribut dan perilaku suatu objek, sedangkan object merupakan hasil konkret atau instansiasi dari class tersebut. Constructor digunakan untuk menginisialisasi nilai awal pada saat objek dibuat, dan method merupakan aksi atau tindakan yang dapat dilakukan oleh objek tersebut. Pemahaman yang baik terhadap keempat konsep ini menjadi fondasi penting sebelum mahasiswa melangkah ke topik OOP yang lebih kompleks seperti pewarisan (inheritance), enkapsulasi, dan polimorfisme.

Oleh karena itu, pada pertemuan pertama praktikum Pemrograman Berorientasi Objek ini, mahasiswa diberikan tugas untuk merancang dan mengimplementasikan sebuah studi kasus sederhana berupa sistem perbankan mini (minibank) menggunakan bahasa pemrograman Java. Melalui studi kasus ini, mahasiswa diharapkan dapat memahami secara langsung bagaimana class, object, constructor, dan method diterapkan dalam sebuah program yang nyata dan fungsional.

1.2 Tujuan

Tujuan yang diharapkan pada praktikum pekan 1 tentang *class*, *objek*, *constructor*, dan *method* yaitu sebagai berikut.

1. Memahami konsep dasar class dan object dalam Pemrograman Berorientasi Objek beserta hubungan keduanya.
2. Memahami fungsi dan cara kerja constructor dalam menginisialisasi nilai awal sebuah objek.
3. Memahami konsep method serta cara mendefinisikan dan memanggil method di dalam sebuah class.

1.3 Manfaat

Manfaat yang didapatkan setelah melakukan praktikum pekan 1 tentang *class*, *objek*, *constructor*, dan *method* yaitu sebagai berikut.

1. Praktikum ini bermanfaat untuk meningkatkan pemahaman konsep dasar OOP secara praktis.
2. Melatih kemampuan mahasiswa dalam merancang class beserta atribut, konstruktor, dan method yang sesuai.

BAB II

PEMBAHASAN

2.1 Pengertian Class, Object, Constructor dan Method

Pada pertemuan minggu pertama praktikum pemrograman berorientasi objek, mahasiswa mempelajari file materi yang telah dibagikan oleh dosen. Selain sumber dari file yang diberi oleh dosen, berikut pengertian dari class, object, constructor dan method.

2.1.1 Class

Class adalah cetak biru atau *blueprint* dari object. Class digunakan hanya untuk membuat kerangka dasar, yang akan kita pakai nanti adalah hasil cetakan dari class, yakni object. Sebagai analogi, class bisa diibaratkan dengan laptop atau notebook, yakni gambaran umum tentang sebuah benda. Didalam pemrograman nanti, contoh class seperti User, Item, Siswa, dll.

Dalam bahasa Java, penulisan class diawali dengan *keyword* class, kemudian diikuti dengan nama dari class tersebut. Aturan penulisan nama class sama seperti aturan penulisan *variabel* di Java (lebih tepatnya aturan identifier), yakni tidak boleh diawali angka dan tidak boleh mengandung spasi. Namun, kebiasaan programmer Java, nama class ditulis menggunakan PascalCase atau UpperCamelCase. Yaitu setiap kata diawali dengan huruf besar, termasuk kata pertama. Contoh class Rekening.

2.1.2 Object

Object atau objek adalah hasil cetak dari class, atau bisa juga disebut hasil konkrit dari class. Masih menggunakan analogi class Rekening, maka object dari class Rekening bisa berupa: rekeningAbdul, rekeningLisa, atau rekeningNabil. Sebuah object dari class Rekening akan memiliki seluruh ciri-ciri

Rekening, termasuk atribut dan method-nya. Proses mencetak object dari class ini disebut dengan instansiasi (atau instantiation).

Dalam bahasa Java, proses instansiasi dilakukan dengan menulis nama class, lalu diikuti nama variabel yang akan menampung object tersebut, tanda sama dengan "=", keyword new dan kembali nama class dengan tambahan tanda kurung "()". Hasil cetakan class akan disimpan dalam variabel untuk selanjutnya dipakai selama penulisan kode program. Variabel inilah yang akan berisi object dari sebuah class.

2.1.3 Constructor dan Atribut

Sebuah Constructor adalah metode khusus yang dijalankan saat objek dari suatu kelas dibuat. Constructor digunakan untuk menginisialisasi objek dengan memberikan nilai awal kepada properti atau atribut dalam objek tersebut. Constructor memiliki nama yang sama dengan nama kelasnya dan tidak mengembalikan nilai (void).

Penggunaan constructor memungkinkan Anda untuk:

- Menginisialisasi nilai-nilai awal objek.
- Melakukan tugas-tugas persiapan saat objek dibuat.
- Memastikan objek dalam keadaan yang valid.

Sedangkan Property (atau kadang juga dengan *atribut* atau *field*) adalah data yang terdapat dalam sebuah class. Melanjutkan analogi tentang *Rekening*, property dari laptop bisa berupa *nomorRekening*, *namaPemilik*, dan *saldo*, dan lain-lain. Jika sudah terbiasa dengan program Java, property ini sebenarnya hanyalah variabel yang terletak di dalam class. Seluruh aturan dan tipe data yang biasa diinput ke dalam variabel, juga bisa diinput ke dalam property. Aturan tata cara penamaan property sama dengan aturan penamaan variabel.

2.1.4 Method

Method adalah tindakan yang bisa dilakukan di dalam class. Jika menggunakan analogi class Mahasiswa, maka contoh method adalah: *tarikTunai*, *setorTunai* atau *membukaRekeningBaru*.

Method pada dasarnya adalah *function* yang berada di dalam class. Seluruh sifat function bisa diterapkan ke dalam method seperti bisa di isi argument/parameter, mengembalikan nilai (dengan keyword *return*), dan lain-lain.

2.2 Implementasi Class, Object, Constructor dan Method

Implementasi dari topik Class, Object, Constructor dan Method pada pertemuan pertama praktikum pemrograman berorientasi objek adalah membuat sebuah minibank sesuai pada modul ajar praktikum pbo pekan 1. Berikut implementasinya dalam kode program java.

2.2.1 Class Rekening

Program berikut merupakan implementasi class Rekening yang berfungsi sebagai *blueprint* atau cetak biru untuk membuat objek rekening bank. Class ini mendefinisikan atribut-atribut yang dimiliki sebuah rekening, konstruktor untuk inisialisasi data saat objek dibuat, serta beberapa method untuk melakukan operasi perbankan sederhana seperti setor tunai, tarik tunai, dan menampilkan informasi rekening. Berikut adalah kode program dari class rekening.

```
1. package pekan1;
2. public class Rekening {
3.     String nomorRekening;
4.     String namaPemilik;
5.     double saldo;
6.
7.     public Rekening(String nomor, String nama, double saldoAwal) {
8.         nomorRekening = nomor;
9.         namaPemilik = nama;
```

```

10.         saldo = saldoAwal;
11.         System.out.println("Rekening atas nama " +
    namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);
12.
13.     }
14.     public void setorTunai (double nominal) {
15.         if (nominal > 10000) {
16.             saldo += nominal;
17.             System.out.println("Setor tunai Rp" +
    nominal + " berhasil. Saldo saat ini: Rp" + saldo);
18.         } else {
19.             System.out.println("Gagal: Nominal
    setor harus lebih dari 0!");
20.         }
21.     }
22.     public void tarikTunai (double nominal) {
23.         if (10000 < nominal && nominal < saldo) {
24.             saldo -= nominal;
25.             System.out.println("Tarik tunai Rp" +
    nominal + " berhasil. Saldo saat ini: Rp" + saldo);
26.         } else {
27.             System.out.println("Gagal: Nominal
    tarik harus lebih dari 10000 atau saldo tidak mencukupi!");
28.         }
29.     }
30.     public void cekInformasi() {
31.         System.out.println("-- INFO REKENING --");
32.         System.out.println("No. Rekening : " +
    nomorRekening);
33.         System.out.println("Nama Pemilik : " +
    namaPemilik);
34.         System.out.println("Saldo Akhir : Rp" +
    saldo);
35.         System.out.println("-----");
36.     }
37. }

```

Class Rekening di atas memiliki tiga atribut, yaitu nomorRekening dan namaPemilik yang bertipe String, serta saldo yang bertipe double. Ketiga atribut ini bersifat sebagai *state* atau kondisi yang melekat pada setiap objek rekening yang dibuat. Konstruktor Rekening (String nomor, String nama, double saldoAwal) digunakan untuk menginisialisasi nilai awal dari ketiga atribut tersebut secara otomatis setiap kali objek baru dibuat menggunakan kata kunci `new`, sekaligus menampilkan pesan konfirmasi bahwa rekening berhasil dibuat.

Selanjutnya, class ini memiliki tiga buah method. Method `setorTunai` (double nominal) digunakan untuk menambahkan nominal ke dalam saldo rekening, dengan syarat nominal yang disetor harus lebih besar dari Rp10.000; jika syarat ini tidak terpenuhi, program akan menampilkan pesan kegagalan. Method `tarikTunai` (double nominal) berfungsi untuk mengurangi saldo rekening, dengan dua syarat validasi yaitu nominal penarikan harus lebih besar dari Rp10.000 dan tidak boleh melebihi saldo yang tersedia, sehingga mencegah terjadinya saldo negatif. Terakhir, method `cekInformasi` () digunakan untuk menampilkan seluruh informasi rekening seperti nomor rekening, nama pemilik, dan saldo akhir dalam format yang rapi.

Melalui class ini, dapat dilihat penerapan konsep dasar OOP, yaitu atribut sebagai representasi data, konstruktor sebagai inisialisasi objek, dan method sebagai representasi perilaku atau aksi yang dapat dilakukan oleh objek Rekening. Selain itu, validasi logika pada method `setorTunai` () dan `tarikTunai` () menunjukkan bagaimana method dapat mengontrol perubahan pada atribut (enkapsulasi sederhana), sehingga data saldo tidak berubah secara sembarangan tanpa melalui aturan bisnis yang telah ditentukan.

2.2.2 Class Main

Program berikut merupakan class `Main` yang berfungsi sebagai program utama (*driver class*) untuk menjalankan dan menguji class `Rekening` yang telah dibuat sebelumnya. Program ini mengimplementasikan sistem perbankan mini berbasis menu interaktif menggunakan `Scanner`, di mana pengguna dapat membuka rekening baru, melakukan setor tunai, tarik tunai, serta melihat informasi rekening melalui perulangan `while` dan struktur percabangan `switch-case`. Berikut adalah kode program dari class `main`.

```
1. package pekan1;
2. import java.util.Scanner;
3.
4. public class Main {
5.
```

```

6. public static void main(String[] args) {
7.     Scanner input = new Scanner (System.in);
8.     Rekening akunAktif = null; // objek belum diinisiasi
9.     boolean isRunning = true;
10.
11.
12.     System.out.println("==== SISTEM PERBANKAN
    MINI ===");
13.
14.     while (isRunning) {
15.         System.out.println("\nMenu Utama:");
16.         System.out.println("1. Buka Rekening
    Baru");
17.         System.out.println("2. Setor Tunai");
18.         System.out.println("3. Tarik Tunai");
19.         System.out.println("4. Cek Informasi
    Rekening");
20.         System.out.println("0. Keluar");
21.         System.out.println("Pilih Menu:");
22.
23.         if (!input.hasNextInt()) {
24.             System.out.println("Error:
    Pilihan menu harus berupa angka!");
25.             input.next();
26.             continue;
27.         }
28.
29.         int pilihan = input.nextInt();
30.         input.nextLine(); //Membersihkan
    Buffer Enter
31.
32.         switch (pilihan) {
33.             case 1:
34.                 System.out.print("Masukkan No
    Rekening: ");
35.                 String no = input.nextLine();
36.                 System.out.print("Masukkan Nama
    Pemilik: ");
37.                 String nama= input.nextLine();
38.                 System.out.print("Masukkan Saldo Awal:
    ");
39.                 if (!input.hasNextDouble()) {
40.                     System.out.println("Error:
    Saldo awal harus berupa angka! Pembuatan rekening dibatalkan.");
41.                     input.next();
42.                     break;
43.                 }
44.                 double saldo = input.nextDouble();
45.
46.
47.                 //Intansiasi Object / Menjalankan
    Constructor
48.                 akunAktif = new Rekening(no, nama,
    saldo);

```

```

49.                 break;
50.
51.                 case 2:
52.                     if (akunAktif == null) {
53.                         System.out.println("Error: Mohon Maaf, Anda belum memiliki nomor
rekening!");
54.                     } else {
55.                         System.out.print("Masukkan Nominal Setor: ");
56.                         double setor =
input.nextDouble();
57.                         akunAktif.setorTunai(setor);
58.                     }
59.                     break;
60.
61.                 case 3:
62.                     if (akunAktif == null) {
63.                         System.out.println("Error: Mohon Maaf, Anda belum memiliki nomor
rekening!");
64.                     } else {
65.                         System.out.print("Masukkan Nominal Tarik: ");
66.                         double Tarik =
input.nextDouble();
67.                         akunAktif.tarikTunai(Tarik);
68.                     }
69.                     break;
70.
71.                 case 4:
72.                     if (akunAktif == null) {
73.                         System.out.println("Error: Anda bekum membuka rekening");
74.                     } else {
75.                         akunAktif.cekInformasi();
76.                     }
77.                     break;
78.
79.                 case 0:
80.                     isRunning = false;
81.                     System.out.println("Sistem
ditutup. Terima Kasih!");
82.                     break;
83.
84.                 default:
85.                     System.out.println("Pililhan Tidak Valid");
86.                 }
87.             }
88.             input.close();

```

```

89.         }
90.     }

```

Pada awal program, dibuat objek Scanner untuk membaca input dari pengguna, sebuah variabel objek akunAktif bertipe Rekening yang diinisialisasi dengan nilai null (menandakan belum ada rekening yang dibuat), dan variabel isRunning bertipe boolean yang berfungsi sebagai kondisi untuk menjaga program tetap berjalan di dalam perulangan while. Selama isRunning bernilai true, program akan terus menampilkan menu utama kepada pengguna secara berulang.

Program juga dilengkapi dengan validasi input menggunakan input.hasNextInt() untuk memastikan pilihan menu yang dimasukkan berupa angka. Struktur switch-case kemudian digunakan untuk mengarahkan alur program sesuai pilihan menu yang dipilih. Pada case 1, program meminta input nomor rekening, nama pemilik, dan saldo awal dari pengguna, lalu melakukan instansiasi objek baru dari class Rekening menggunakan kata kunci new, yang secara otomatis akan menjalankan konstruktor dan menyimpan objek tersebut ke dalam variabel akunAktif.

Pada case 2, case 3, dan case 4, program terlebih dahulu memeriksa apakah akunAktif masih bernilai null atau tidak, jika null, artinya pengguna belum membuat rekening sehingga program akan menampilkan pesan error, dan jika sudah terisi objek, program akan memanggil method yang sesuai dari objek akunAktif, yaitu setorTunai(), tarikTunai(), atau cekInformasi(). Sementara itu, case 0 digunakan untuk mengubah nilai isRunning menjadi false, sehingga perulangan while akan berhenti dan program selesai dijalankan. Secara keseluruhan, class Main ini menunjukkan bagaimana sebuah objek dari class Rekening diinstansiasi dan digunakan dalam program nyata melalui pemanggilan method-method yang telah didefinisikan sebelumnya.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan pada pertemuan pertama mata kuliah Pemrograman Berorientasi Objek, dapat disimpulkan bahwa class merupakan cetak biru atau blueprint yang mendefinisikan atribut dan method yang akan dimiliki oleh suatu objek, sedangkan object adalah hasil konkret atau instansiasi dari class tersebut yang memiliki data dan perilakunya masing-masing. Constructor berperan penting dalam menginisialisasi nilai awal atribut suatu objek secara otomatis pada saat objek dibuat menggunakan kata kunci new, sedangkan method merupakan tindakan atau fungsi yang berada di dalam class dan dapat dipanggil untuk menjalankan suatu proses tertentu.

3.2 Saran

Untuk praktikum selanjutnya, disarankan agar lebih banyak berlatih membuat studi kasus lain dengan class dan atribut yang berbeda, sehingga pemahaman terhadap konsep class, object, constructor, dan method menjadi lebih mendalam dan tidak hanya bergantung pada satu contoh kasus saja.

Selain itu, penambahan validasi input yang lebih lengkap, seperti pengecekan format nomor rekening atau penanganan kesalahan input secara lebih menyeluruh, dapat menjadi bahan pengembangan program pada praktikum berikutnya agar program yang dibuat semakin robust dan mendekati aplikasi nyata.

DAFTAR PUSTAKA

[1] Andre, “Tutorial OOP Java Part 2: Pengertian Class, Object, Property dan Method,”. [Daring]. Tersedia pada: Tutorial OOP Java: Pengertian Class, Object, Property & Method [Diakses: 10-September-2026].

[2] Ahmad Yoga Bachtiar Teguh Suseno, “Learn OOP in Java Programming Language #06 | Understanding Constructors,”. [Daring]. Tersedia pada: [Belajar OOP Bahasa Pemrograman Java #06 | Pengertian Constructor — Artikel Minarsih TECH](#) [Diakses: 10-September-2026].

LAPORAN TUGAS PRAKTIKUM

Pemograman Berorientasi Objek

Pekan 1 (*Class, Object, Constructor dan Method*)

Dosen Pengampu:

Nurfiah, S.ST, M.Kom.

Asisten Praktikum:

M Dawi Syauqi



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

Deskripsi Tugas

1. Implementasi Fitur Tarik Tunai

- Tambahkan method tarikTunai(double nominal) ke dalam class Rekening.
- Aturan: Saldo tidak boleh menjadi minus dan minimal nominal Tarik tunai adalah 10.000. Jika pengguna mencoba menarik nominal yang lebih besar dari saldo saat ini atau nominal kecil dari 10.000, program tidak boleh crash/error, melainkan menampilkan peringatan ke konsol: "Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp[nominal]" atau "Transaksi Gagal: Minimal nominal penarikan 10.000".
- Terapkan method ini dengan menu nomor 3 pada class Main.

2. Tantangan Multi-Akun

Saat ini, aplikasi Main hanya dapat menyimpan 1 akun (Rekening akunAktif = null;). Jika pengguna memilih menu "Buka Rekening Baru" untuk kedua kalinya, akun lama akan tertimpa.

- Ubah arsitektur penyimpanan di Main.java menggunakan struktur data Collection. (Gunakan ArrayList).
- Ubah logika menu "Buka Rekening": Setiap kali pengguna membuka rekening baru, objek tersebut ditambahkan ke dalam ArrayList.
- (Bonus): Tambahkan menu baru untuk "Ganti Akun", di mana sistem akan meminta nomor rekening, kemudian mencari objek yang cocok di dalam ArrayList untuk dijadikan akunAktif

PEMBAHASAN TUGAS

1. Implementasi Fitur Tarik Tunai

Implementasi fitur Tarik tunai dilakukan pada class rekening, method dibuat hampir sama dengan method setor tunai, hanya saja berbeda pada pengurangan dan penambahan saldo. Namun, pada method tarikTunai juga harus memenuhi beberapa syarat yaitu minimal nominal penarikan adalah 10.000 dan tidak boleh melebihi dari saldo yang ada. Berikut adalah kode program method yang sesuai dan berdasarkan syarat tugas.

```
1. public void tarikTunai(double nominal) {  
2.     if (nominal < 10000) {  
3.         System.out.println("Transaksi Gagal: Minimal nominal  
penarikan 10.000");  
4.     }  
5.     else if (nominal > saldo) {  
6.         System.out.println("Transaksi Gagal: Saldo tidak  
mencukupi. Saldo Anda: Rp" + saldo);  
7.     }  
8.     else {  
9.         saldo -= nominal;  
10.        System.out.println("Tarik tunai Rp" + nominal + "  
berhasil. Saldo saat ini: Rp" + saldo);  
11.    }  
12. }
```

Method tarikTunai menerima satu parameter bernama nominal bertipe data double yang merepresentasikan jumlah uang yang ingin ditarik dari rekening. Method tarikTunai menerima nilai nominal penarikan dan memprosesnya melalui tiga tahap pengecekan. Blok if pertama memastikan nominal tidak kurang dari 10.000; jika kurang, penarikan dibatalkan dengan peringatan minimal penarikan. Selanjutnya, blok else if mengecek apakah nominal melebihi saldo; jika ya, transaksi dibatalkan dan informasi saldo saat ini ditampilkan. Jika lolos kedua validasi tersebut, blok else akan mengurangi saldo sebesar nominal yang ditarik lalu mencetak konfirmasi bahwa penarikan berhasil beserta sisa saldo terbaru.

2. Tantangan Multi-akun

Pada tugas tantangan multi akun, saya menambahkan atau mengubah kode program sebelumnya. Seperti penambahan method, penggunaan array list dan lainnya. Berikut adalah kode program dari dua class. Yaitu class Rekening dan class Main.

a. Class Rekening

Berikut kode program lengkap dari class Rekening

```

1. package pekan1;
2. public class Rekening {
3.     String nomorRekening;
4.     String namaPemilik;
5.     double saldo;
6.
7.     public Rekening(String nomor, String nama, double saldoAwal) {
8.         nomorRekening = nomor;
9.         namaPemilik = nama;
10.        saldo = saldoAwal;
11.        System.out.println("Rekening atas nama " +
    namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);
12.
13.    }
14.
15.    public String getNomorRekening() {
16.        return nomorRekening;
17.    }
18.
19.    public void setorTunai (double nominal) {
20.        if (nominal > 10000) {
21.            saldo += nominal;
22.            System.out.println("Setor tunai Rp" +
    nominal + " berhasil. Saldo saat ini: Rp" + saldo);
23.        } else {
24.            System.out.println("Gagal: Nominal setor
    harus lebih dari 0!");
25.        }
26.    }
27.    public void tarikTunai(double nominal) {
28.        if (nominal < 10000) {
29.            System.out.println("Transaksi Gagal: Minimal
    nominal penarikan 10.000");
30.        }
31.        else if (nominal > saldo) {
32.            System.out.println("Transaksi Gagal: Saldo tidak
    mencukupi. Saldo Anda: Rp" + saldo);
33.        }
34.        else {

```

```

35.         saldo -= nominal;
36.         System.out.println("Tarik tunai Rp" + nominal + "
berhasil. Saldo saat ini: Rp" + saldo);
37.     }
38. }
39.     public void cekInformasi() {
40.         System.out.println("-- INFO REKENING --");
41.         System.out.println("No. Rekening : " +
nomorRekening);
42.         System.out.println("Nama Pemilik : " +
namaPemilik);
43.         System.out.println("Saldo Akhir : Rp" + saldo);
44.         System.out.println("-----");
45.     }
46. }

```

Pada kelas Rekening.java, perubahan dilakukan dengan menambahkan *method getter* `getNomorRekening()` yang mengembalikan nilai atribut `nomorRekening`. Penambahan ini bertujuan untuk menerapkan prinsip enkapsulasi, sehingga atribut nomor rekening dapat dibaca dan diakses oleh kelas lain secara aman tanpa mengubah nilainya langsung. Akses pembacaan ini sangat diperlukan oleh kelas utama saat melakukan pemindaian dan pencarian akun di dalam daftar penyimpanan.

b. Class Main

Berikut kode program lengkap dari class Main

```

1. package pekan1;
2. import java.util.Scanner;
3. import java.util.ArrayList;
4.
5. public class Main {
6.
7.     public static void main(String[] args) {
8.         Scanner input = new Scanner (System.in);
9.
10.        ArrayList<Rekening> daftarRekening = new
ArrayList<>();
11.        Rekening akunAktif = null; // objek belum
diinisiasi
12.        boolean isRunning = true;
13.
14.
15.        System.out.println("==== SISTEM PERBANKAN MINI
====");
16.

```

```

17.         while (isRunning) {
18.             System.out.println("\nMenu Utama:");
19.             System.out.println("1. Buka Rekening Baru");
20.             System.out.println("2. Setor Tunai");
21.             System.out.println("3. Tarik Tunai");
22.             System.out.println("4. Cek Informasi
    Rekening");
23.             System.out.println("5. Ganti Akun");
24.             System.out.println("0. Keluar");
25.             System.out.println("Pilih Menu:");
26.
27.             if (!input.hasNextInt()) {
28.                 System.out.println("Error: Pilihan
    menu harus berupa angka!");
29.                 input.next();
30.                 continue;
31.             }
32.
33.             int pilihan = input.nextInt();
34.             input.nextLine(); //Membersihkan Buffer
    Enter
35.
36.             switch (pilihan) {
37.                 case 1:
38.                     System.out.print("Masukkan No Rekening: ");
39.                     String no = input.nextLine();
40.                     System.out.print("Masukkan Nama Pemilik: ");
41.                     String nama= input.nextLine();
42.                     System.out.print("Masukkan Saldo Awal: ");
43.                     if (!input.hasNextDouble()) {
44.                         System.out.println("Error: Saldo awal
    harus berupa angka! Pembuatan rekening dibatalkan.");
45.                         input.next();
46.                         break;
47.                     }
48.                     double saldo = input.nextDouble();
49.                     input.nextLine();
50.
51.                     //Intansiasi Object / Menjalankan
    Constructor
52.                     akunAktif = new Rekening(no, nama, saldo);
53.
54.                     //simpan ke dalam arraylist
55.                     daftarRekening.add(akunAktif);
56.                     break;
57.
58.                 case 2:
59.                     if (akunAktif == null) {
60.                         System.out.println("Error:
    Mohon Maaf, Anda belum memiliki nomor rekening!");
61.                     } else {
62.                         System.out.print("Masukkan
    Nominal Setor: ");

```

```

63.         input.nextDouble();
64.             akunAktif.setorTunai(setor);
65.         }
66.         break;
67.
68.         case 3:
69.             if (akunAktif == null) {
70.                 System.out.println("Error:
Mohon Maaf, Anda belum memiliki nomor rekening!");
71.             } else {
72.                 System.out.print("Masukkan
Nominal Tarik: ");
73.                 double Tarik =
input.nextDouble();
74.                 akunAktif.tarikTunai(Tarik);
75.             }
76.             break;
77.
78.         case 4:
79.             if (akunAktif == null) {
80.                 System.out.println("Error: Anda
bekum membuka rekening");
81.             } else {
82.                 akunAktif.cekInformasi();
83.             }
84.             break;
85.
86.         case 5:
87.             if (daftarRekening.isEmpty()) {
88.                 System.out.println("Error:
Belum ada rekening yang terdaftar!");
89.             } else {
90.                 System.out.print("Masukkan
Nomor Rekening yang ingin diaktifkan: ");
91.                 String noCari =
input.nextLine();
92.
93.                 Rekening ditemukan = null;
94.                 for (Rekening r :
daftarRekening) {
95.                     if
(r.getNomorRekening().contentEquals(noCari)) {
96.                         ditemukan = r;
97.                         break;
98.                     }
99.                 }
100.                 if (ditemukan != null) {
101.                     akunAktif = ditemukan;
102.
103.                     System.out.println("Sukses: Akun aktif berhasil diubah ke no
rekening " + akunAktif.getNomorRekening());
104.                 } else {

```

```

104.         System.out.println("Error: Nomor rekening tidak ditemukan!");
105.     }
106.     }
107.     break;
108.
109.     case 0:
110.         isRunning = false;
111.         System.out.println("Sistem ditutup.
Terima Kasih!");
112.         break;
113.
114.     default:
115.         System.out.println("Pililhan
Tidak Valid");
116.     }
117. }
118. input.close();
119. }
120. }

```

Pada kelas Main.java, pembaruan utama berfokus pada perubahan arsitektur penyimpanan dari satu objek tunggal menjadi struktur data dinamis berjenis `ArrayList<Rekening>`. Perubahan diawali dengan mengimpor pustaka `java.util.ArrayList` serta mendeklarasikan objek `daftarRekening` untuk menampung banyak akun tanpa menimpa data lama. Pada pilihan menu pertama (case 1), ditambahkan instruksi `daftarRekening.add(akunAktif)` agar setiap rekening baru yang diinstansiasi otomatis tersimpan ke dalam *list*, diikuti pembersihan *buffer scanner* menggunakan `input.nextLine()`. Terakhir, ditambahkan opsi menu baru pada case 5 ("Ganti Akun") yang memuat logika validasi status `ArrayList`, perulangan *for-each* untuk mencocokkan nomor rekening inputan menggunakan `contentEquals()`, serta perubahan acuan variabel `akunAktif` ke objek rekening yang berhasil ditemukan.